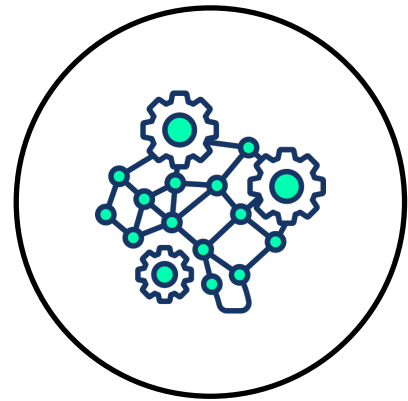


Deep Generative Modeling

University of Manitoba

Assignment 2 (ECE-7650)



Author:

Md. Masud Mazumder

Term: Winter 26

Student ID: 8056333

Instructors:

Ian Jeffrey

Associate Professor

Dept. of Electrical and Computer Engineering

University of Manitoba, Canada

Vahab Khoshdel

Assistant Professor

Dept. of Electrical and Computer Engineering

University of Manitoba, Canada

Code Available: [GitHub Repository](#)

Date: March 1, 2026

Declaration

I, Md. Masud Mazumder, attest that the work I am submitting is my own work and that it has not been copied/plagiarized from online or other sources. Any sourced material used for completing this work has been properly cited.

Signature

Md. Masud Mazumder

Contents

A Problem 1: Variational Autoencoder	4
A.1 Task 1	4
A.2 Task 2	5
A.3 Task 3	8
A.4 Task 4	9
A.5 Task 5	10
B Problem 2: Variational Autoencoder Application	16
B.1 Task 1	16
C Additional Discussion Questions	19
C.1 Question 1	19

List of Figures

1	True sample generation from Bernoulli distribution.	5
2	True sample generation from a Gaussian distribution.	6
3	Plotting generated samples (images).	7
4	True samples generated from trained VAE using the Bernoulli distribution.	7
5	True samples generated from trained VAE using the Gaussian distribution.	7
6	Reconstruction and KL loss curves.	9
7	Latent space cluster visualization.	10
8	3D Latent space reconstruction visualization.	11
9	2D latent space reconstruction visualization.	12
10	Code snippet for latent space clustering	13
11	Code snippet for latent space reconstruction	14
12	2D vs 3D latent space loss curves.	15
13	Code snippet for linear interpolation between two digit class centers.	17
14	Linear interpolation between digit classes 1 to 9.	18

A Problem 1: Variational Autoencoder

A.1 Task 1

Problem statement: What additional steps are required to use the VAE to generate a sample?

Answer: The output of the VAE (more specifically, decoder output) in the provided notebook is the mean of the conditional distribution $p(x|z)$. However, a true generative sample requires sampling from the full probabilistic model rather than simply using this mean.

To generate a true sample from a trained VAE, the following additional steps are required:

1. **Sample from the prior distribution**

Since the latent prior is defined as

$$p(z) = \mathcal{N}(0, \mathbf{I})$$

we must first draw a random latent vector $z \sim \mathcal{N}(0, \mathbf{I})$.

2. **Pass the sampled latent vector through the decoder**

The decoder outputs the mean of the likelihood distribution $p(x|z)$. This gives us the parameter (mean) of a Gaussian distribution (according to the class lecture) in the data space.

3. **Sample from the output distribution**

The decoder defines $p(x|z) = N(f(z; \phi), c\mathbf{I})$. Therefore, instead of directly using the mean $f(z; \phi)$, we should sample from the appropriate probability distribution. Based on the class lecture, the normal distribution and corresponding formula are as follows:

$$x = f(z; \phi) + \sigma\epsilon, \quad \epsilon \sim \mathcal{N}(0, \mathbf{I})$$

In summary, the additional step required for true generation is sampling from the probabilistic output distribution, rather than simply using the decoder's mean output.

A.2 Task 2

Problem statement: Augment the code to generate and display 10 random samples from a trained VAE.

Solution: To generate true samples from a trained VAE, we first sample a latent vector from the prior distribution $\mathcal{N}(0, \mathbf{I})$ of the same dimensionality as the latent space. This latent vector is then passed through the decoder to obtain the output parameters. However, generating the final image requires interpreting the decoder output probabilistically rather than using it directly. The correct sampling strategy depends on the reconstruction loss used during training.

In this task, I consider two possible cases: Binary Cross-Entropy (BCE), which is used in the provided implementation, and Mean Squared Error (MSE).

- **BCE Loss:** BCE assumes that each pixel follows a Bernoulli distribution, meaning the decoder output represents the probability that a pixel is “on” (value = 1), rather than the pixel value itself. Therefore, to obtain a true generative sample, we should draw pixel values from this probability distribution instead of using the output directly. This is achieved by sampling from a Bernoulli distribution (equivalently, a Binomial distribution with $n = 1$) using the decoder’s output probabilities. Figure 1 illustrates this sampling strategy.



```
1 num_samples = 10
2
3 # Sampling latent vectors from the prior distribution
4 z = np.random.normal(size=(num_samples, latent_dim))
5
6 # Get decoder output (these are probabilities / means)
7 generated_probs = decoder.predict(z)
8
9 # Sample from Bernoulli distribution
10 generated_samples = np.random.binomial(1, generated_probs)
11
12 # Display generated samples
13 plot_sample_images(num_samples, generated_samples)
```

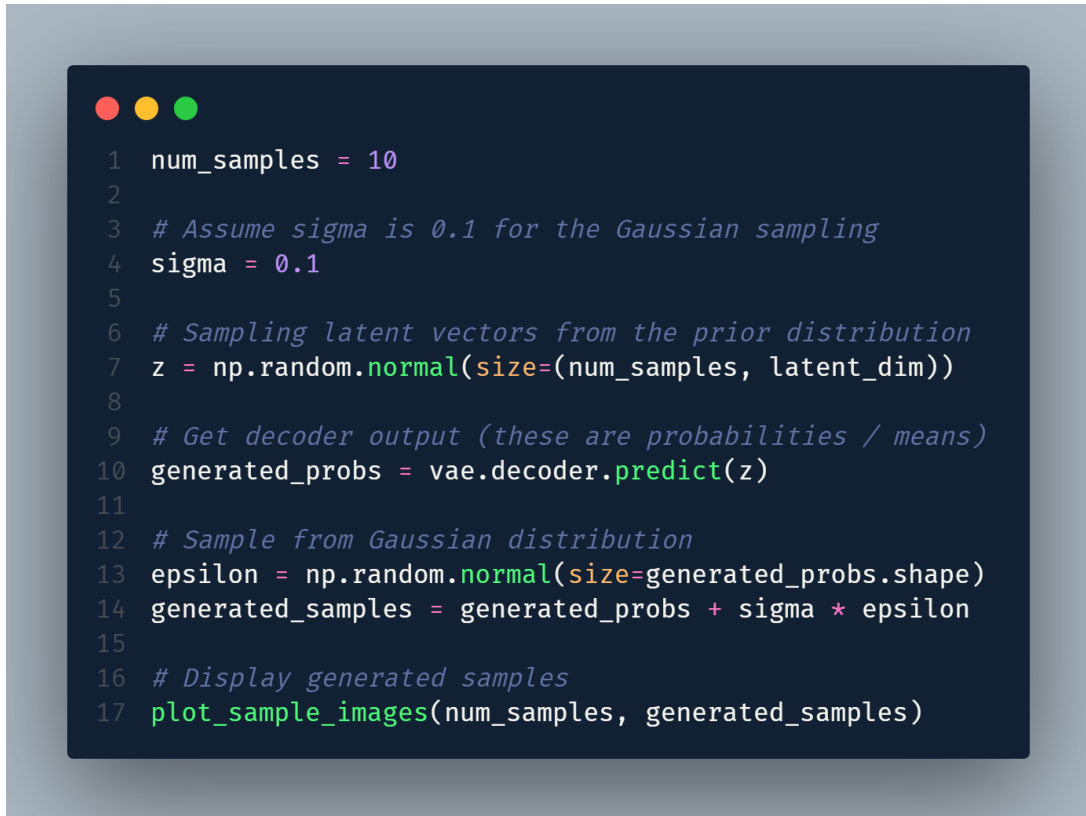
Figure 1: True sample generation from Bernoulli distribution.

- **MSE Loss:** If MSE loss is used, the probabilistic interpretation changes. In VAEs, MSE corresponds to assuming that the pixel values follow a Gaussian distribution with fixed variance. In this case, the decoder output represents the

mean of that distribution. A true sample can therefore be generated by adding Gaussian noise to this mean. This is implemented using the formula:

$$\text{true_sample} = \text{mean} + \text{sigma} * \text{epsilon}$$

where $\text{epsilon} \sim \mathcal{N}(0, \mathbf{I})$. Here, sigma is treated as a constant, and I set sigma = 0.1. Figure 2 shows this sampling procedure.

A code editor window with a dark blue background and three colored window control buttons (red, yellow, green) in the top left corner. The code is written in a light blue monospace font. It defines the number of samples as 10, sets sigma to 0.1, samples latent vectors z from a normal distribution, gets decoder output probabilities, samples epsilon from a normal distribution, and finally generates true samples by adding sigma * epsilon to the probabilities. The code ends with a function call to plot the generated samples.

```
1 num_samples = 10
2
3 # Assume sigma is 0.1 for the Gaussian sampling
4 sigma = 0.1
5
6 # Sampling latent vectors from the prior distribution
7 z = np.random.normal(size=(num_samples, latent_dim))
8
9 # Get decoder output (these are probabilities / means)
10 generated_probs = vae.decoder.predict(z)
11
12 # Sample from Gaussian distribution
13 epsilon = np.random.normal(size=generated_probs.shape)
14 generated_samples = generated_probs + sigma * epsilon
15
16 # Display generated samples
17 plot_sample_images(num_samples, generated_samples)
```

Figure 2: True sample generation from a Gaussian distribution.

Finally, the generated samples are visualized using the plotting function shown in Figure 3. Figure 4 and Figure 5 show the generated true samples from the trained VAE using the Bernoulli distribution and the Gaussian distribution, respectively. ***While generating true samples for the Gaussian distribution, I retrained the model with the MSE loss function.***

Note: According to the discussion in the class, the output probabilities of the decoder while using the BCE loss seemed to be considered as the mean for the true generated image. In this task, however, I chose to follow the probabilistic interpretation of these outputs.

```

1  # Displaying generated samples
2  def plot_sample_images(num_samples, generated_samples):
3      plt.figure(figsize=(num_samples, 2))
4      for i in range(num_samples):
5          ax = plt.subplot(1, num_samples, i + 1)
6          plt.imshow(generated_samples[i].squeeze(), cmap="gray")
7          plt.axis("off")
8      plt.show()

```

Figure 3: Plotting generated samples (images).



Figure 4: True samples generated from trained VAE using the Bernoulli distribution.



Figure 5: True samples generated from trained VAE using the Gaussian distribution.

A.3 Task 3

Problem statement: What conclusions can you draw about the generative quality of the VAE when it is used in a “truly generative” fashion (i.e., sampling the normal distribution output of the model)? Is it common practice to use a VAE in this manner? Or does one usually just use the mean? Support your answer with references.

Answer: If we examine the sampling equation from the normal distribution,

$$image = mean + sigma * epsilon$$

the decoder output corresponds to the mean $\mathbf{mean} = \mathbf{f}(\mathbf{z})$, while the term $\mathbf{sigma} * \mathbf{epsilon}$ represents stochastic variation where $\epsilon \sim \mathcal{N}(0, \mathbf{I})$. Although this term may appear to introduce arbitrary noise, it actually reflects the uncertainty captured by the learned likelihood distribution $p(x|z)$.

In practice, however, this stochasticity often manifests as pixel-level perturbations rather than meaningful structural variation. As observed in Figure 5, sampling does not significantly alter semantic features such as digit shape; instead, it introduces local variability that appears as blur or noise. This behavior is well documented in the literature and arises from the Gaussian likelihood assumption used in standard VAEs (Kingma and Welling [2013]). Specifically, Kingma and Welling define the generative process as:

$$p(x|z) = \mathcal{N}(f(z), c\mathbf{I})$$

which implies that the decoder output corresponds to the conditional expectation

$$\mathbf{E}[x|z] = f(z)$$

and that sampling produces variations around this expectation rather than fundamentally new structural realizations.

While sampling is theoretically necessary for treating the VAE as a fully generative model, it is not commonly used when high perceptual quality is desired. As discussed by Prince [2023] (Chapter 17), sampling is primarily required when the goal is to generate new data by drawing from the learned distribution. In many practical applications, such as reconstruction, denoising, and inpainting, using the mean output directly often yields more visually coherent results.

Conversely, sampling becomes important in tasks that explicitly require diversity or uncertainty modeling, such as data simulation, augmentation, and probabilistic generation. This distinction aligns with observations by Doersch [2016], who notes that VAEs are optimized for likelihood-based objectives rather than perceptual sharpness.

Overall, when used in a truly generative manner by sampling from the decoder’s output distribution, VAEs produce diverse outputs but often with reduced visual sharpness due to the Gaussian likelihood assumption. In practice, it is more common to use the decoder mean for tasks requiring high-quality reconstructions, while sampling is primarily used in applications that benefit from variability or uncertainty modeling.

A.4 Task 4

Problem statement: During training, what is the behavior of the two loss terms? Does one dominate? Should it? Why or why not?

Answer: The VAE loss consists of two competing terms:

- the reconstruction loss and
- the KL divergence.

The reconstruction term encourages accurate data reconstruction, while the KL divergence regularizes the latent distribution by pushing it toward the prior $p(z)$.

During early training, the reconstruction loss is typically large and dominates, as the model has not yet learned meaningful representations of the data. The KL term is initially small because the encoder outputs are often close to the prior distribution. As training progresses, reconstruction improves, and the KL divergence increases as the latent space becomes structured. Figure 6 shows this phenomenon while training the VAE for the MNIST dataset.

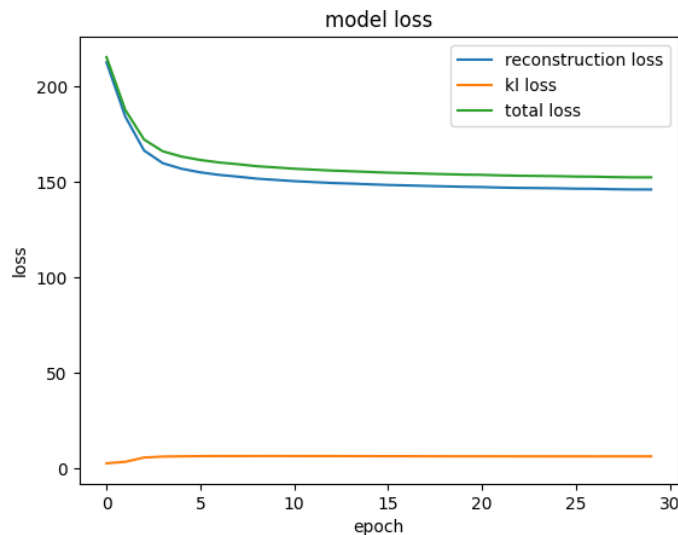


Figure 6: Reconstruction and KL loss curves.

Ideally, neither term should dominate. If the reconstruction loss dominates, the model may ignore the prior and overfit the training data, leading to poor generative performance. Conversely, if the KL term dominates, the encoder may collapse toward the prior distribution, causing the latent variables to carry little information about the input. This phenomenon, known as posterior collapse, results in degraded reconstructions.

A balanced interaction between the two terms is therefore essential. The reconstruction term ensures fidelity to the data, while the KL divergence enforces a smooth and generative latent space. Both are necessary for learning a useful probabilistic representation.

A.5 Task 5

Problem statement: Increase the latent space from 2 to 3 and re-train the VAE. Visualize the results by showing appropriately selected slices through the latent space. Do you think there is any advantage to a larger latent space for this particular problem? Why or why not?

Answer: In this task, the latent dimensionality of the VAE was increased from 2 to 3, and the model was retrained. No major changes in the original code were necessary for this, except for changing the value of the variable `latent_dim` from 2 to 3. Since direct visualization of a 3D latent space is difficult, appropriately selected 2D slices were examined by fixing one latent dimension and varying the other two.

Figure 7 shows the latent space cluster separation, and Figure 8 shows the reconstructions obtained from different slices of the latent space generated by the code snippet in Figure 10 and Figure 11, respectively. Both of these figures were generated by fixing one dimension while varying the others. The clustering and reconstruction results indicate that the additional latent dimension allowed for greater flexibility in representing the data. Compared to the 2D latent space reconstruction (Figure 9), the 3D latent space (Figure 8) exhibited smoother transitions between digit styles and more continuous variations in the generated samples. The latent structure also appeared more stable across different slices, suggesting that the model had increased capacity to encode meaningful features.

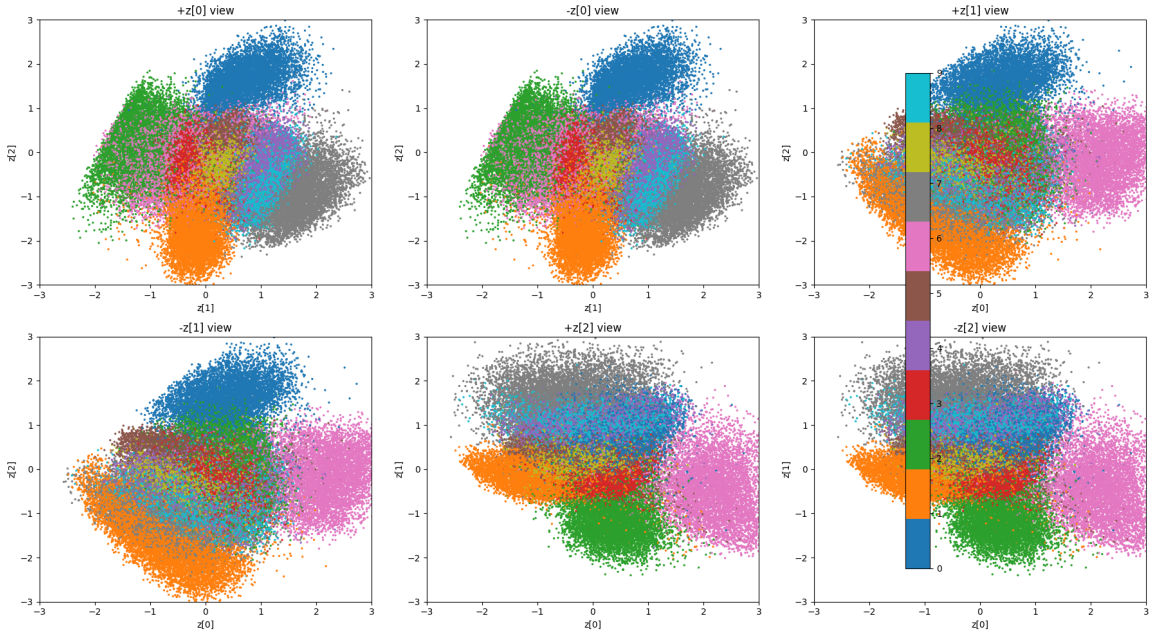


Figure 7: Latent space cluster visualization.

When compared with the 2D latent space and its corresponding reconstructions, some improvement can be observed in the 3D latent representation. However, the improvement is modest for this particular problem. The loss curves comparison in

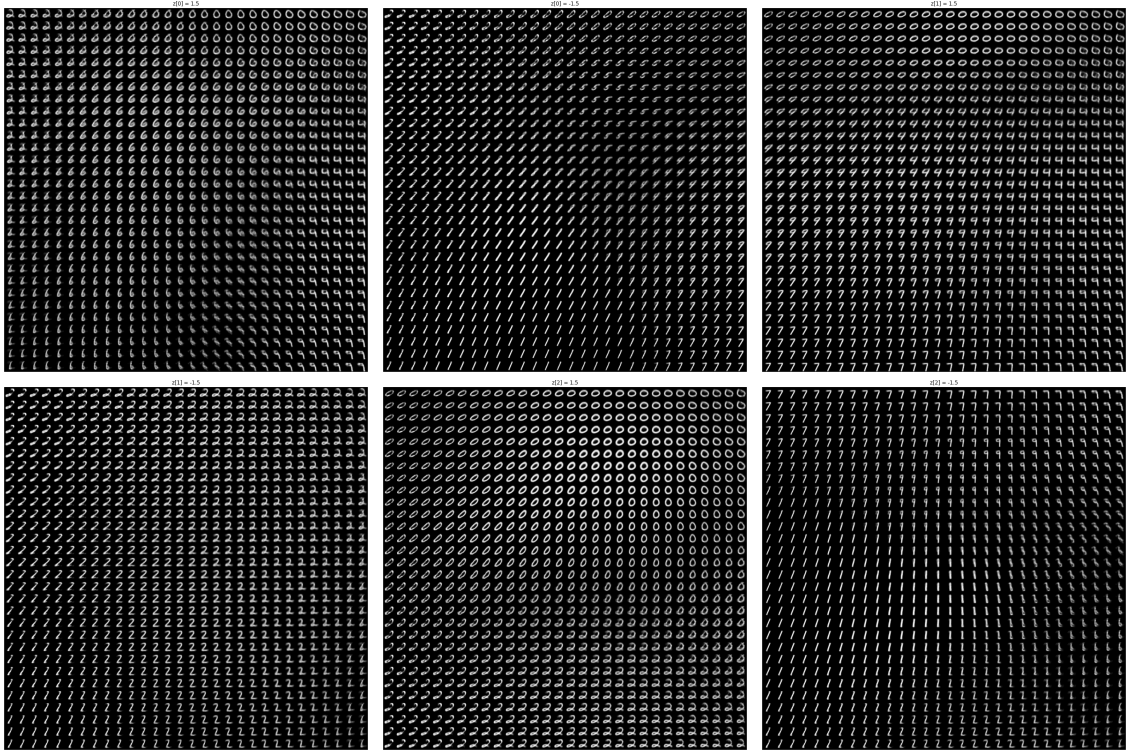


Figure 8: 3D Latent space reconstruction visualization.

Figure 12 also exhibits this phenomenon, indicating a very low improvement in the loss in 3D latent space representation. This is likely because MNIST digits are relatively simple and low-dimensional in nature, meaning that a 2D latent space is already sufficient to capture most of the variation in the dataset. The additional dimension primarily distributes the representation more smoothly rather than introducing fundamentally new structural diversity.

From a probabilistic modeling perspective, a larger latent space increases the representational capacity of the model, allowing it to capture more complex variations in the data (Kingma and Welling [2013]). As discussed by Prince [2023], latent variables provide a compact representation of underlying factors of variation, and increasing dimensionality can improve modeling flexibility.

Nevertheless, for simple datasets such as MNIST, the added complexity does not necessarily translate into significant perceptual gains. A higher-dimensional latent space may become more advantageous for datasets with greater variability or more complex structure.

In conclusion, while increasing the latent space dimensionality improves representational capacity and smoothness of latent organization, the practical benefit for MNIST is limited. The trade-off between model complexity and performance suggests that a 2D latent space is already sufficient for this problem.

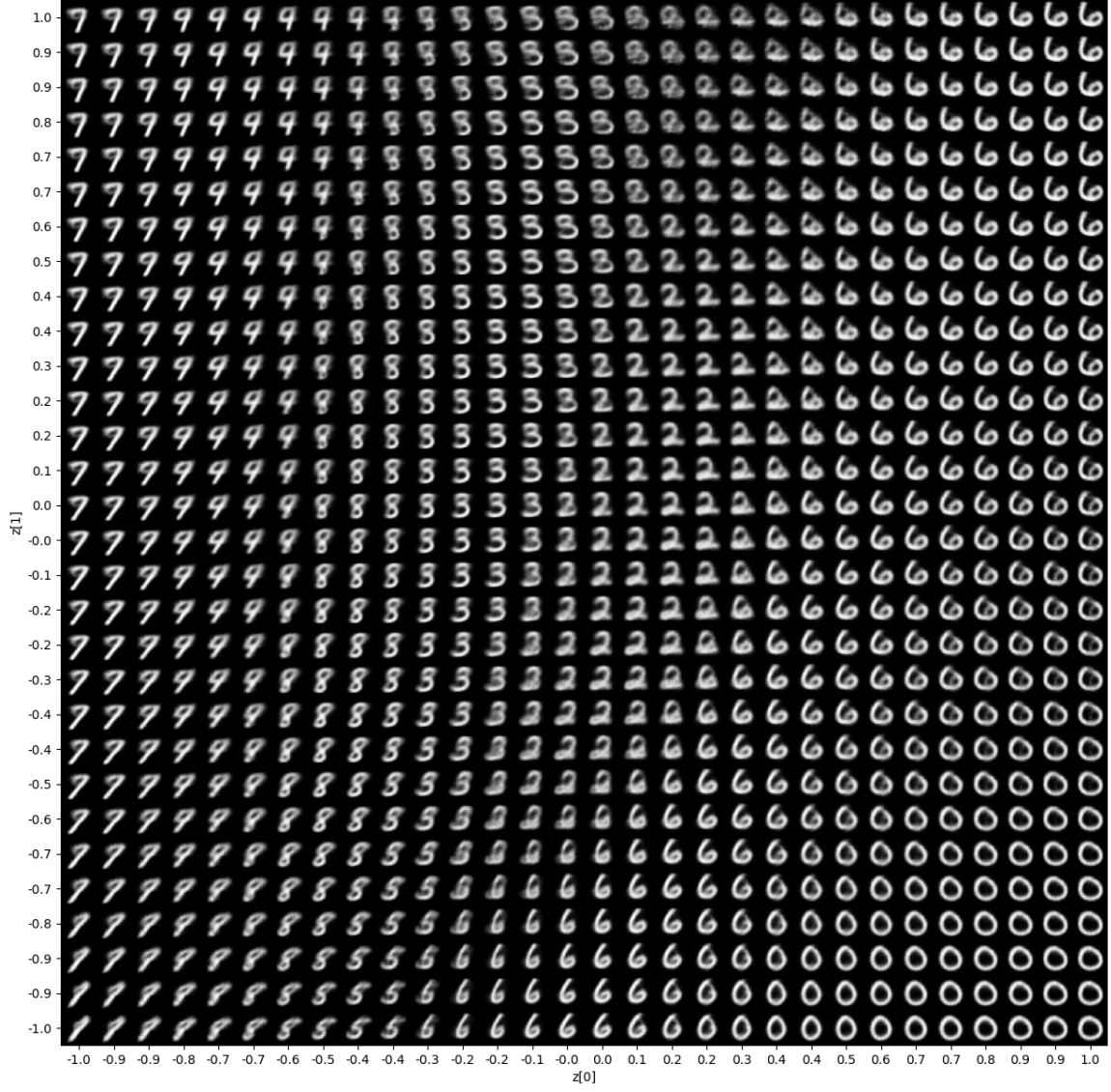


Figure 9: 2D latent space reconstruction visualization.

```

1  def plot_3d_latent_sides(x_train, labels, lim=3):
2      z_mean, z_log_var, z = vae.encoder.predict(x_train, verbose=0)
3      fig, axes = plt.subplots(2, 3, figsize=(18, 10))
4
5      plots = [
6          (1, 2, "z[1]", "z[2]", "+z[0] view"),
7          (1, 2, "z[1]", "z[2]", "-z[0] view"),
8
9          (0, 2, "z[0]", "z[2]", "+z[1] view"),
10         (0, 2, "z[0]", "z[2]", "-z[1] view"),
11
12         (0, 1, "z[0]", "z[1]", "+z[2] view"),
13         (0, 1, "z[0]", "z[1]", "-z[2] view"),
14     ]
15
16     for ax, (i, j, xl, yl, title) in zip(axes.flat, plots):
17         sc = ax.scatter(
18             z[:, i],
19             z[:, j],
20             c=labels,
21             s=2,
22             cmap="tab10"
23         )
24         ax.set_xlabel(xl)
25         ax.set_ylabel(yl)
26         ax.set_title(title)
27         ax.set_xlim(-lim, lim)
28         ax.set_ylim(-lim, lim)
29
30     fig.colorbar(sc, ax=axes.ravel().tolist())
31     plt.tight_layout()
32     plt.show()
33
34 plot_3d_latent_sides(mnist_digits, mnist_digit_labels)

```

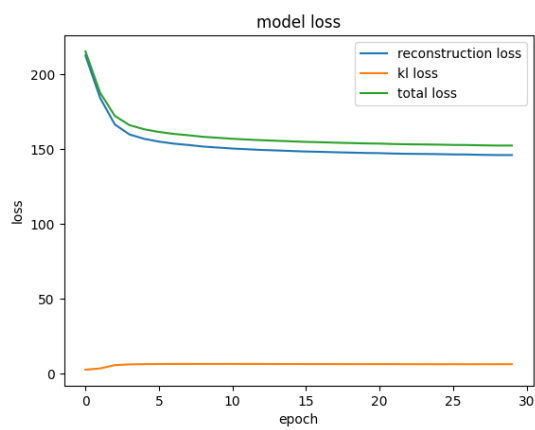
Figure 10: Code snippet for latent space clustering

```

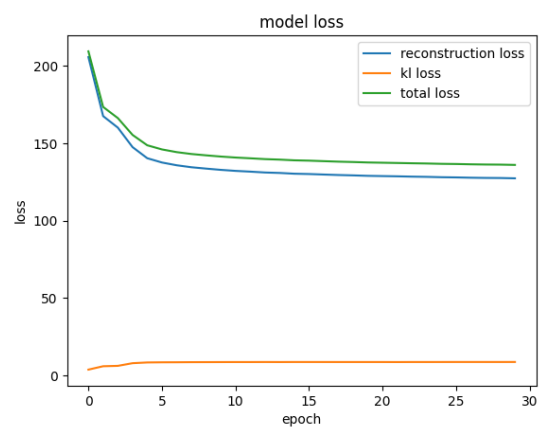
1 def plot_latent_plane(decoder, fixed_dim, fixed_value, n=30, scale=1.5):
2     digit_size = 28
3     figure = np.zeros((digit_size * n, digit_size * n))
4
5     grid = np.linspace(-scale, scale, n)
6
7     for i, yi in enumerate(grid[::-1]):
8         for j, xi in enumerate(grid):
9
10            z = np.zeros((1, 3))
11
12            coords = [xi, yi]
13            idx = 0
14
15            for k in range(3):
16                if k == fixed_dim:
17                    z[0, k] = fixed_value
18                else:
19                    z[0, k] = coords[idx]
20                    idx += 1
21
22            x_decoded = decoder.predict(z, verbose=0)
23            digit = x_decoded[0].reshape(28, 28)
24
25            figure[
26                i*digit_size:(i+1)*digit_size,
27                j*digit_size:(j+1)*digit_size
28            ] = digit
29
30            plt.figure(figsize=(15, 15))
31            plt.imshow(figure, cmap="gray")
32            plt.title(f"z[{fixed_dim}] = {fixed_value}")
33            plt.axis("off")
34            plt.show()
35
36 c = 1.5
37
38 plot_latent_plane(vae.decoder, fixed_dim=0, fixed_value=+c)
39 plot_latent_plane(vae.decoder, fixed_dim=0, fixed_value=-c)
40
41 plot_latent_plane(vae.decoder, fixed_dim=1, fixed_value=+c)
42 plot_latent_plane(vae.decoder, fixed_dim=1, fixed_value=-c)
43
44 plot_latent_plane(vae.decoder, fixed_dim=2, fixed_value=+c)
45 plot_latent_plane(vae.decoder, fixed_dim=2, fixed_value=-c)

```

Figure 11: Code snippet for latent space reconstruction



(a) 2D latent space loss curve.



(b) 3D latent space loss curve.

Figure 12: 2D vs 3D latent space loss curves.

B Problem 2: Variational Autoencoder Application

B.1 Task 1

Problem statement: Demonstrate latent space interpolation for a VAE of your choice (the given VAE is sufficient). Pay special attention to how you choose your two latent space points (some approaches are more interesting than others). Provide a summary of your implementation and how it performed.

Solution: Latent space interpolation in VAE involves encoding two data points (e.g., images x_1, x_2) into latent vectors (z_1, z_2) , generating intermediate latent representations using a linear path

$$z_{interp} = (1 - \alpha)z_1 + \alpha z_2, \quad \alpha \in [0, 1]$$

and decoding these intermediate vectors to produce a smooth, semantic transition, such as transforming one handwritten digit into another in this case.

There are several possible approaches for selecting the two latent points for interpolation. For example:

- Interpolation between two randomly sampled latent vectors
- Interpolation between latent representations of individual samples
- Interpolation between class centers

Among these options, interpolating between class centers is particularly informative, as it captures the overall semantic structure of each class rather than the idiosyncrasies of individual samples. Therefore, in this task, I perform interpolation between the latent centers of two digit classes (**digits 1 and 9**).

The overall procedure from selecting two latent vectors (z_1 and z_2), representing two digit classes, to interpolating through a linear path followed by visualization is summarized below:

1. Select two classes of digits of interest (1 and 9).
2. Extract all samples belonging to each class from the training dataset.
3. Encode these samples using the VAE encoder to obtain their latent representations.
4. Compute the mean latent vector for each class.
5. Perform linear interpolation between the two class centers.
6. Decode the interpolated latent vectors using the decoder.

7. Visualize the reconstructed outputs.

Figure 13 shows the code snippet for this overall process. The resulting sequence of decoded outputs (Figure 14) demonstrates a smooth and gradual transformation between the digits 1 and 9. Here, Interpolation 1 represents the mean of digit class 1, and Interpolation 20 represents the mean of digit class 9. These outputs show that intermediate samples exhibit blended characteristics of the two endpoints rather than abrupt changes, indicating that the VAE has learned a continuous and structured latent representation of the data.

```
1 # Latent space interpolation between two class centers
2 # Let's choose two digit classes, for example, 1 and 9
3 digit_class_1 = 1
4 digit_class_2 = 9
5
6 # We can use the encoder to compute the mean latent vector for each class
7 _, _, z = vae.encoder.predict(x_train, verbose=0)
8 class_1_indices = np.where(y_train == digit_class_1)[0]
9 class_2_indices = np.where(y_train == digit_class_2)[0]
10
11 # Compute the mean latent vector for each class
12 class_1_mean = np.mean(z[class_1_indices], axis=0)
13 class_2_mean = np.mean(z[class_2_indices], axis=0)
14
15 # Now we interpolate between the two mean vectors and decode the results
16 n_samples = 20
17 interpolated = np.array(
18     [
19         class_1_mean * (1 - alpha) + class_2_mean * alpha
20         for alpha in np.linspace(0, 1, n_samples)
21     ]
22 )
23 decoded = vae.decoder.predict(interpolated, verbose=0)
24
25 # Display the interpolated images
26 plt.figure(figsize=(20, 4))
27 for i in range(n_samples):
28     plt.subplot(2, n_samples // 2, i + 1)
29     plt.imshow(decoded[i].reshape(28, 28), cmap="Greys_r")
30     plt.title(f"Interpolation {i+1}")
31     plt.axis("off")
```

Figure 13: Code snippet for linear interpolation between two digit class centers.

This approach produces more meaningful transitions than interpolating between random latent vectors, which may lie in low-density regions of the latent space and yield less interpretable results. By interpolating between class centers, the transformation reflects semantic variation at the class level rather than minor stylistic differences.

Overall, the interpolation performed well, producing plausible intermediate samples that maintained digit-like structure throughout the transition. However, some intermediate images appeared slightly blurred, which is consistent with the Gaussian likelihood assumption commonly used in VAEs (Doersch [2016]).

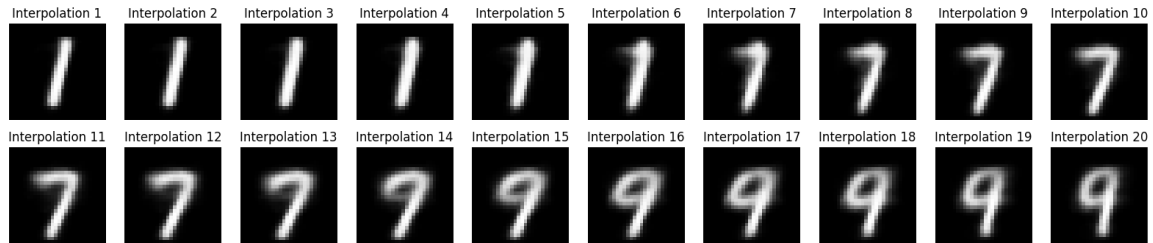


Figure 14: Linear interpolation between digit classes 1 to 9.

C Additional Discussion Questions

C.1 Question 1

AEs and VAEs both use a latent representation, but VAEs provide a distributional interpretation of the output. Outside of generation, what tasks are there that AEs are better at than VAEs? Vice versa? Support your answer with references.

Answer: Autoencoders (AEs) and Variational Autoencoders (VAEs) both aim to learn latent representations of data, but they excel at different tasks outside of data generation due to their fundamentally different objectives. AEs learn deterministic encodings optimized purely for reconstruction accuracy, while VAEs learn probabilistic latent representations constrained by a prior distribution. As a result, AEs are generally better at tasks requiring precise reconstruction, whereas VAEs are more effective in tasks requiring structured representations or uncertainty modeling.

Tasks Where AEs Perform Better:

Standard AEs learn deterministic encodings and lack the regularization constraint (KL divergence) used in VAEs. They focus solely on minimizing reconstruction error, making them superior in specific scenarios. Some of them are listed below:

- **Reconstruction/Dimensionality Reduction/Compression:** Because AEs are not constrained by a Kullback-Leibler (KL) divergence term that forces the latent space into a specific distribution, they can achieve lower reconstruction error and preserve more detail than VAEs, which may produce smoother and less blurred outputs (Kingma and Welling [2013]).
- **Data Denoising:** While both can denoise, vanilla AEs are often highly effective at learning to remove specific types of noise from a dataset by mapping corrupted inputs directly back to known, clean outputs. Denoising Autoencoders (DAEs) are specifically designed to remove noise and are widely used (Vincent et al. [2008]).
- **Anomaly Detection:** AEs are widely used for anomaly detection because they reconstruct normal data well but fail on abnormal samples (Aytekin et al. [2018]).
- **Feature Extraction (Deterministic):** In scenarios where the goal is to extract efficient, non-probabilistic features for a classification model, deterministic AEs can be more straightforward and efficient to train, avoiding complex optimization of the ELBO loss (Bengio et al. [2013]).

Tasks Where VAEs Perform Better:

Because VAEs enforce a latent prior

$$q(z|x) \sim \mathcal{N}(0, \mathbf{I}),$$

they learn structured latent spaces. Therefore, VAEs are better for tasks requiring a structured, continuous latent space that can be navigated or where uncertainty is crucial. A few of such common tasks are listed below:

- **Representation Learning/Interpolation:** Because VAEs enforce a prior distribution on latent variables:

$$q(z|x) \sim \mathcal{N}(0, \mathbf{I})$$

they learn continuous and smooth latent space. [Kingma and Welling \[2013\]](#) show that this structure supports meaningful interpolation between two points and representation learning.

- **Data Augmentation:** Because the encoder outputs a distribution, we can sample from the latent space near an original input to generate diverse, synthetic, but realistic variations of the training data ([Doersch \[2016\]](#)).
- **Uncertainty Modeling:** VAEs explicitly model uncertainty through distributions rather than point estimates. VAEs capture variability in the data, making them useful for probabilistic reasoning ([Doersch \[2016\]](#)).
- **Clustering and Structure Analysis:** Because the latent space in VAEs is forced to follow a probability distribution, it creates organized, compact clusters of data, which is better for finding semantic structures in data without labels ([Higgins et al. \[2017\]](#)).
- **Probabilistic Anomaly Detection:** Because VAEs learn a smooth, probabilistic distribution of the normal data (via the KL divergence penalty), they are often better at identifying outliers. They can detect instances that fall into low-probability regions of the latent space, whereas AEs might overfit and reconstruct anomalies well ([An and Cho \[2015\]](#)).

To summarize, AEs are more effective for deterministic reconstruction-driven tasks such as denoising and compression. VAEs are better suited for structured representation learning, uncertainty modeling, and probabilistic analysis. Thus, while VAEs extend AEs with generative capability, their key advantage lies in learning smooth and semantically meaningful latent spaces.

References

- Jinwon An and Sungzoon Cho. Variational autoencoder based anomaly detection using reconstruction probability. *Special lecture on IE*, 2(1):1–18, 2015.
- Caglar Aytekin, Xingyang Ni, Francesco Cricri, and Emre Aksu. Clustering and unsupervised anomaly detection with l2 normalized deep auto-encoder representations. In *2018 International Joint Conference on Neural Networks (IJCNN)*, pages 1–6. IEEE, 2018.
- Yoshua Bengio, Aaron Courville, and Pascal Vincent. Representation learning: A review and new perspectives. *IEEE transactions on pattern analysis and machine intelligence*, 35(8):1798–1828, 2013.
- Carl Doersch. Tutorial on variational autoencoders. *arXiv preprint arXiv:1606.05908*, 2016.
- Irina Higgins et al. beta-vae: Learning basic visual concepts with a constrained variational framework. *ICLR*, 2017.
- Diederik P Kingma and Max Welling. Auto-encoding variational bayes. *arXiv preprint arXiv:1312.6114*, 2013.
- Simon JD Prince. *Understanding deep learning*. MIT press, 2023.
- Pascal Vincent, Hugo Larochelle, Yoshua Bengio, and Pierre-Antoine Manzagol. Extracting and composing robust features with denoising autoencoders. In *Proceedings of the 25th international conference on Machine learning*, pages 1096–1103, 2008.